CHAPTER 9

Advanced Gem Management

We now have a working development environment, all based on Docker. However, there's one area that is worth a bit more thought: gem management.

Up until now, to install or update gems, we've simply been rebuilding the image for our Rails app. This works because `bundle install` is one of the steps in our `Dockerfile`. However, as we'll see in a moment, there's a slight downside to this approach compared to what we're used to when managing gems in a non-Dockerized environment (you may have already spotted this).

In this chapter, we'll explore an alternative approach to managing our gems that attempts to avoid the drawback by making different trade-offs (it's more complicated for a start).

Whether you stick with the simple approach we've been using so far, or this new approach, is completely up to you. I will present the technique, explain the trade-offs, and then you can choose based on your needs and preferences. Deal?

The Downside to Our Existing Approach

Our current approach for managing gems is to rebuild the image any time we need to update our gems. However, you may have noticed that any time we change our `Gemfile`, even if it's just adding a single gem, *all* our gems have to be reinstalled from scratch. As a result, updating our gems takes longer than we're typically used to.

Why does this happen?

Bundler and Docker images are both trying to achieve the same goal of ensuring a consistent environment, but they achieve it in different ways. Docker's image-build process breaks some of Bundler's key assumptions, which means it doesn't quite work how we're used to.